```
    p = mp.Process(target=worker, args=
(i,MasterNode,counter,params))    5
    p.start()
    processes.append(p)
for p in processes:
6
    p.join()
for p in processes:
7
    p.terminate()

print(counter.value,processes[1].exitcode)
8
```

- ***1*** Creates a global, shared actor-critic model
- ***2*** The shared_memory() method will allow the parameters of the model to be shared across processes rather than being copied.
- ***3*** Sets up a list to store the instantiated processes
- ***4*** A shared global counter using multiprocessing's built-in shared object. The 'i' parameter indicates the type is integer.
- ***5*** Starts a new process that runs the worker function
- ***6*** "Joins" each process to wait for it to finish before returning to the main process
- ***7*** Makes sure each process is terminated
- ***8*** Prints the global counter value and the first process's exit code (which should be 0)

This is exactly the same setup we had when we demonstrated how to split up an array across multiple processes, except this time we're going to be running a function called `worker` that will run our CartPole reinforcement learning algorithm.

Next we'll define the worker function, which will run a single agent in an instance of the CartPole environment.

Listing 5.6. The main training loop

```
def worker(t, worker_model, counter, params):
    worker_env = gym.make("CartPole-v1")
    worker_env.reset()
    worker_opt = optim.Adam(lr=1e-
4,params=worker_model.parameters())     1
```